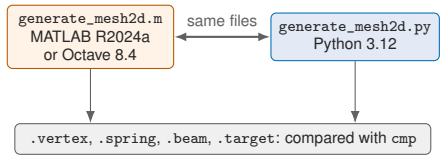
APPENDIX A: MATLAB AND PYTHON HELPER CODES

Every maintained MATLAB script of the series beside its Python version: the course exercises, the readers, the structure generators; how to run them, how they were checked against each other, and what to watch for when translating



26 MATLAB scripts in the examples (`legacy/` aside), each with a Python version that writes the same files (to round-off), computes the same numbers or reads the same data

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Appendix A

Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this appendix you should be able to:

1. find the Python version of any MATLAB script of the series, and the chapter that uses it;
2. run the course's four small exercises in MATLAB, Octave or Python, and read what they show;
3. check that two versions of a generator write the same structure files, and know which differences are round-off;
4. translate a MATLAB script to Python without the usual slips (indices from 1 or 0, ranges, array shapes, plots on a cluster).

> Use whichever language you have for the 26 MATLAB scripts of the examples: each has a Python version, and the pairs were run against each other in Octave 8.4, MATLAB R2024a and Python 3.12 (Octave has no `h5read`, which `samrai_dumps.m` needs). The other 40 Python scripts of the examples have no MATLAB version (slide 4).

WHERE THE MATLAB SCRIPTS ARE (1/3): WITH THEIR PYTHON VERSIONS

folder	MATLAB	Python
`examples/support/`	`heat_equation.m`,	`heat_equation.py`,
`matlab-exercises`	`stiff_ode_example.m`,	`stiff_ode_example.py`,
	`test_IB2d_IBAMR_Geometry.m`,	`test_ib_geometry.py`,
	`read_Vertex_Points_`	`read_vertex_points_`
	`and_Plot_Them.m`	`and_plot_them.py`
`examples/analysis`	`read_hier_data.m`, `read_input.m`,	`hier_data.py` (with `read_input`),
	`leaf_forces.m`, `samrai_dumps.m`,	`leaf_forces.py`, `samrai_dumps.py`,
	`read_vtk_legacy.m`, `read_exodus.m`,	`read_vtk_legacy.py`,
	`read_curve.m`	`read_exodus.py`, `read_curve.py`
`examples/IB/*`	`generate_mesh2d.m` (01, 03, 04, 06, 07, 09–12), `Channel_Flow.m` (02), `generate_mesh3d.m` (08)	`generate_mesh2d.py`, `channel_flow.py`, `generate_mesh3d.py`
`examples/ConstraintIB/*`	`Cylinder2d.m` (01, 02), `eel2d_straightswimmer.m` (03)	`generate_cylinder2d.py`, `generate_eel2d.py`
`examples/IBFE/` `01-tethered-plate/extras`	`Beam_IBFE_Mesh2D.m`	`beam_ibfe_mesh2d.py`

26 `.m` files in all, outside `examples/legacy`: 4 course exercises, 7 readers, 15 structure generators. The scripts with no MATLAB version, and the older `.m` files, are on the next two slides.

The folders' `README.md` files (October 2026).

WHERE THE MATLAB SCRIPTS ARE (2/3): THE SCRIPTS WITH NO MATLAB VERSION

- Python only (no MATLAB version in this repository):
  - in `analysis`: VisIt's scripts (`visit_3d.py`, chapter 27; `visit_queries.py`, 29; `visit_movie_frames.py`, 30; `resample_export.py`, 33), the movie tools (`make_movie.py`, `movie_clean.py`, chapter 30), `flow_rate.py`, `hier_data_h5.py` and `piv_to_vtk.py` (chapter 33), `vtk_for_planktos.py` (chapter 35);
  - IB/11's `make_inputs.py`, IB/13's `vessel_walls.py` and `make_input.py`, IBFE/05's `fix_orientation.py`;
  - `examples/Planktos` (chapters 34 and 35), and the IB2d folders' Python (chapters 4c–4f, 19 files): generators, drivers and update functions ported from IB2d's own MATLAB examples (their originals stay in IB2d), the stability lab's `sweep.py` and `analyze.py`, the muscle-model scripts and `svg_to_vertex.py`.

The folders' `README.md` files and the scripts' headers (October 2026).

WHERE THE MATLAB SCRIPTS ARE (3/3): THE COURSE VERSIONS, AND THE ORIGINALS

- The course versions: `examples/legacy` keeps 19 more `.m` files, the structure generators of the UNC and University of Arizona course folders as they were (`generate_mesh2d.m`, `generate_mesh3d.m`, `Channel_Flow.m`, `Beam_IBFE_Mesh2D.m`, and `generate_mesh2d_plate.m` of a 2020 plate example that is not part of the series). They have no Python versions; 17 of them have a maintained successor among the examples of slide 3 (the seven identical copies of `Beam_IBFE_Mesh2D.m` are kept once, in IBFE/01's `extras`), and the two generators of the 2020 example have none (chapter 36).
- The originals of the course exercises are in `originals/MATLAB codes/`: `heat_equation.m`, `stiff_ode_example.m`, and `test_IB_Geometry_and_Springs/` with the two plotting scripts and the rubber band they were run on.

The folders' `README.md` files and the scripts' headers (October 2026); chapter 36 on `legacy/`.

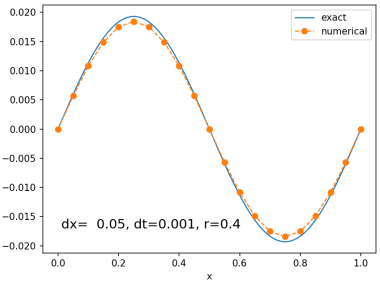
RUNNING THEM: MATLAB, OCTAVE OR PYTHON

```
matlab -batch "heat_equation(19,100)"        # MATLAB, from the script's folder
octave-cli --eval "heat_equation(19,100)"    # Octave: the same call, no desktop
python3 heat_equation.py 19 100              # Python 3 with NumPy and Matplotlib
python3 heat_equation.py 19 100 --show       # ... and open the plot in a window
```

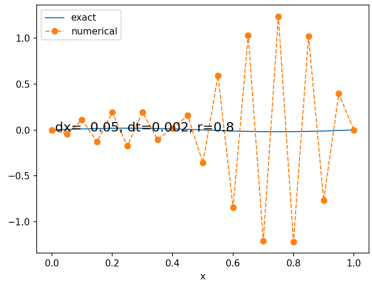
- Versions used for this series: MATLAB R2024a, Octave 8.4, Python 3.12 (NumPy, Matplotlib; the readers also h5py for SAMRAI's HDF5 files and netCDF4 for Exodus).
- The Python versions save their plots as PNG files, so they also run on a cluster without a display; `--show` opens a window as well.
- Octave runs every `.m` file of `analysis` (with its `netcdf` package for `read_exodus.m`) except `samrai_dumps.m`, which needs MATLAB's `h5read`; MATLAB R2024a runs it (slide 16).
- With a display, `octave-cli` still opens a plot window (fltk); without one it draws with gnuplot (a text plot and a warning) and `heat_equation.m` still writes `Example4_1.eps`; with neither a display nor gnuplot, `heat_equation.m`, `stiff_ode_example.m` and `Channel_Flow.m` stop at their first plot.
- The ConstraintIB generators had their `plot` lines commented out so that `octave-cli` runs them on any machine: with a display or gnuplot Octave would draw the plot (or print it as text) and carry on, as it does for `Channel_Flow.m` and the exercises, but with neither, the first plot stops the script before the file is written.

examples/support/matlab-exercises/README.md; examples/analysis/README.md; chapter 0.2 (versions).

THE HEAT EQUATION, AND THE STABILITY OF AN EXPLICIT SCHEME (1/2)



`heat_equation.py 19 100`: $r = 0.4$, stable.



`heat_equation.py 19 50`: $r = 0.8$, unstable.

- $u_t = u_{xx}$ on $0 < x < 1$, $u(0) = u(1) = 0$, $u(x, 0) = \sin 2\pi x$, to $t = 0.1$, against the exact $e^{-4\pi^2 t} \sin 2\pi x$. `N` interior points ($\Delta x = 1/(N + 1)$), `T` steps ($\Delta t = 0.1/T$).
- Stable only for $r = \Delta t/\Delta x^2 \leq 1/2$: largest error 8.7×10^{-4} at $r = 0.4$ everywhere; at $r = 0.8$, 1.25 in Python on Windows (the picture), 1.50 in Python on Linux, 1.30 in MATLAB R2024a and 5.3 in Octave 8.4, while the solution itself is 0.019.

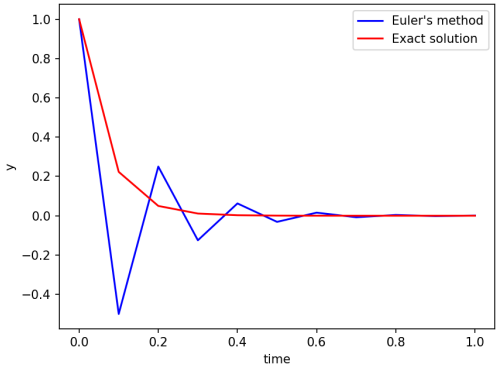
`examples/support/matlab-exercises/heat_equation.py` (run for this appendix); chapters 5 and 4d use the same scheme as the model of an explicit time-step limit: IBAMR and IB2d treat the viscous term implicitly to avoid this one, and it is the explicit spring force whose limit they meet (chapter 8a).

THE HEAT EQUATION, AND THE STABILITY OF AN EXPLICIT SCHEME (2/2): ROUND-OFF GROWS

- The size differs because the growth starts from round-off at any $r > 1/2$: the grid's $\sin 2\pi x$ is a mode the scheme damps ($\times 0.92$ per step at $r = 0.8$; with 60-digit arithmetic the error stays 2.3×10^{-3}), and the zigzag mode, seeded by rounding errors near 10^{-16}, grows $\times 2.18$ per step, 10^{17} in 50 steps. Just above $1/2$ ($r = 0.57$, 19 70) it grows $\times 1.27$ per step, 2×10^{7} in 70 steps, and is still too small to see.

The runs for this appendix: Python 3.12 on Windows and on Linux, MATLAB R2024a, Octave 8.4, and a 60-digit run (mpmath) of the same scheme.

A STIFF EQUATION AND EULER'S METHOD

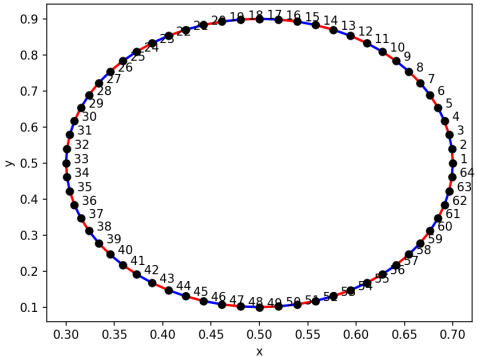


`stiff_ode_example.py 0.1 1`: stable, but the sign flips every step.

- $y' = -15y$, $y(0) = 1$, exact e^{-15t}. Each Euler step multiplies y by $1 - 15\Delta t$.
- $\Delta t < 1/15$: smooth decay (`0.05`); $1/15 < \Delta t < 2/15$: the sign alternates while the size decays (`0.1`, the picture); $\Delta t > 2/15 = 0.133$: it grows (`0.15`).
- The point for the IB method: stiff springs make the equations of motion stiff, and an explicit step must shrink with the stiffness: here, with no inertia, $\Delta t < 2/15$ ($\propto 1/\lambda$ for $y' = -\lambda y$); for the springs of chapter 8a's rule, which move fluid mass, $\Delta t \propto 1/\sqrt{k}$ (four times the stiffness, half the step).
- Changes from the course version: `zeros(N)` made $N \times N$ matrices (the plot drew N curves), and the time grid stopped one step short of `tfinal`.

`examples/support/matlab-exercises/stiff_ode_example.py` and its README; chapters 4d, 5, 8a and 16.

CHECKING A STRUCTURE BEFORE RUNNING IT



`test_ib_geometry.py rubberband`: 64 points with their numbers, the springs in alternating colours.

- `test_ib_geometry.py NAME [--ibamr]` (Python) / `test_IB2d_IBAMR_Geometry.m` (MATLAB: the file names and `flag_IBAMR = 0` or `1` are set at the top of the script): plots `NAME.vertex` with the point numbers, then the `NAME.spring` connections, so that a missing or crossed spring shows before the run does.
- IB2d numbers points from 1 (MATLAB) and IBAMR from 0 (C++): `--ibamr` labels from 0 and reads the springs that way. pyIB2d's files are 0-based too (chapter 4c).
- `read_Vertex_Points_and_Plot_Them.m` / `read_vertex_points_and_plot_them.py NAME`: the points alone.
- The 64-point IB2d rubber band in the folder is what they were run on: "64 points, 64 springs" in Python 3.12, MATLAB R2024a and Octave 8.4. Octave needed a fix (October 2026): its `textscan` does not end a row at the one-number count line, so the course versions misread the files there; both now read the count with `fgetl`.

`examples/support/matlab-exercises` (run for this appendix); chapters 4a, 6 and 9a on structure files.

THE STRUCTURE GENERATORS (1/3): THE EXAMPLES OF CHAPTERS 3–9

example	MATLAB / Python	checked
IB/01 rubber band	`generate_mesh2d.m` / `.py`	`.spring` identical; the shipped `.vertex` is MATLAB's (R2024a reproduces it exactly), and against it a few dozen of the 1288 coordinates differ in the last digit, by at most 1.4×10^{-17} (44 in Octave 8.4 and in Python 3.12 on Linux, which agree with each other; 32 in Python on Windows): sine and cosine round differently in different math libraries
IB/02 pulsatile channel	`Channel_Flow.m` / `channel_flow.py`	the four files identical to the shipped ones
IB/03 tethered beam	`generate_mesh2d.m` / `.py`	identical files (Octave 8.4, Python 3.12)
IB/04 moving cylinders	`generate_mesh2d.m` / `.py` (new in 2026)	Python and Octave write exactly the shipped files; MATLAB R2024a differs in 9 of 594 coordinates, by 4×10^{-19} m (its sine rounds one angle differently)
IB/06 growing springs, IB/07 moving plates	`generate_mesh2d.m` / `.py`	identical files

The examples' `README.md` files (October 2026).

THE STRUCTURE GENERATORS (2/3): THE EXAMPLES OF CHAPTERS 9B, 10, 12 AND 13

example	MATLAB / Python	checked
IB/08 glycocalyx 3D	`generate_mesh3d.m` / `.py`	`.target` identical; the shipped `.vertex` is MATLAB's (R2024a reproduces it); Octave 8.4 and Python 3.12 on Linux differ from it in the same 160 of the 40,800 coordinates, Python on Windows in 480, all in the last digit ($\leq 1.7 \times 10^{-18}$ m)
IB/09, 10, 12	`generate_mesh2d.m` / `.py`	identical files (IB/12: the course's script with its beam loop fixed, which wrote a 170th beam, `169 170 171`, past the last point under a correct header of 169; the Python version new)
IB/11 heart-tube pump	`generate_mesh2d.m` / `.py`	all 27 files identical between Octave 8.4 and Python 3.12 on one system; the shipped `heart_race_512.vertex` is Python-on-Windows output, and Octave or Python on Linux (101 of its 6,872 coordinates) and MATLAB R2024a (88) differ from it in the last digit ($\leq 6 \times 10^{-17}$ m)

The examples' `README.md` files; every generator run for this appendix in Octave 8.4, MATLAB R2024a and Python 3.12 (Windows and Linux).

THE STRUCTURE GENERATORS (3/3): THE EXAMPLES OF CHAPTERS 14 AND 17, AND THE IBFE MESHES

example	MATLAB / Python	checked
ConstraintIB/01, 02	`Cylinder2d.m` / `generate_cylinder2d.py`	identical byte for byte in Octave 8.4 and Python 3.12, and equal to the file shipped upstream; MATLAB on Windows writes the same numbers with CRLF line endings (`fopen(..., 'wt')`): compare there with `diff --strip-trailing-cr`
ConstraintIB/03	`eel2d_straightswimmer.m` / `generate_eel2d.py`	the same as 01 and 02: identical byte for byte, equal to upstream (CRLF from MATLAB on Windows)
IBFE/01 (extras)	`Beam_IBFE_Mesh2D.m` / `beam_ibfe_mesh2d.py`	the same Triangle `.poly` outline (not used by the example)

The IBFE examples' meshes come from gmsh (IBFE/05, 10), Trelis (IBFE/08) or MATLAB's PDE Toolbox (IBFE/01, 06 and 12 share `IBFE_Mesh2D_128.mat`: the PDE Modeler, then a five-line `fprintf` script printed in IBFE/01's README and in chapter 17, with no `.m` file and no Python version); chapter 18 compares the routes. IB/13's walls come from `vessel_walls.py` (Python only).

CHECKING THAT TWO VERSIONS AGREE (1/2): THE RECIPE

```
cp -r . /tmp/check && cd /tmp/check     # a copy: the mv lines empty the folder
shopt -s nullglob                       # skip the file types an example lacks
octave-cli generate_mesh2d.m && mkdir m && mv *.vertex *.spring *.beam *.target *.mass m/
python3 generate_mesh2d.py   && mkdir p && mv *.vertex *.spring *.beam *.target *.mass p/
for f in m/*; do cmp "$f" "p/$(basename $f)" && echo "same: $f"; done
```

- Same bytes is the best result, and most generators reach it: write numbers with the same format (`fprintf('%1.16e')` and `'%1.16e' % x`) and compute them in the same order.
- Run from the example's folder (IB/01, 03, 04, 06, 07, 09–12). IB/02's, IB/08's, the ConstraintIB and IBFE/01's generators have other names (and IBFE/01's writes a `.poly` file): put their names in the two middle lines.
- Last-digit differences (about 10^{-16} relative, one or two units in the 17th significant digit: at most 1.4×10^{-17} in IB/01, 4×10^{-19} m in IB/04, 1.7×10^{-18} m in IB/08, 6×10^{-17} m in IB/11) are round-off: `sin` and `cos` differ between math libraries, between MATLAB and Octave (IB/01, 04, 08, 11) and between Python on Windows and on Linux (IB/01, 08, 11). Compare such files as numbers, `max(abs(a - b))`, not as text.

The recipe run on IB/01, 03 and 12 for this appendix; the READMEs' check notes.

CHECKING THAT TWO VERSIONS AGREE (2/2): WHAT THE PORTS FOUND

- Anything larger than round-off is a bug in one of them.
- Porting also found bugs in the course scripts, some of which no comparison of outputs shows:
  - a beam loop that ran one triple too far and wrote a 170th beam naming point 171 of a 171-point line (0–170) under a header of 169 (IB/12);
  - `zeros(N)` where a vector was meant, so the plot drew N curves, and a time grid one step short of `tfinal` (the stiff ODE);
  - a comment that gave $\sin 3\pi x$ as the initial condition (the heat equation);
  - a count line that Octave's `textscan` read together with the first point (the geometry checkers: in Octave one stopped, the other plotted each y against the next point's x);
  - in `samrai_dumps.m`, written against the Python reader before MATLAB was at hand: `h5read` returns the variable names as strings since R2020a, and a variable named `cell` hid the function `cell`.
- Only the beam line and the time grid change what the scripts write; the others change what they read, plot or say.

The READMEs' "Changes from the course versions"; IB/12's README and the course's `examples/legacy/UNC-Examples/8-AddingMass-Spirder/spider2d_256.beam` (its 170th line); the fixes of October 2026, run in Octave 8.4, MATLAB R2024a and Python 3.12.

THE READERS: THE SAME DATA IN MATLAB, OCTAVE AND PYTHON

MATLAB / Python	reads	chapter
`read_hier_data.m` / `hier_data.py`	the `X.*` and `F.*` files (PETSc ASCII) of `hier_data_IB2d`	11
`read_input.m` / `hier_data.read_input`	a number from an input file, e.g. `DT`	11
`leaf_forces.m` / `leaf_forces.py`	IB/03's drag and lift: minus the sum of the forces on all 103 points of the leaf (the springs' and beams' forces cancel, so it is the pull of its two target points), and the ends' displacement	11
`samrai_dumps.m` / `samrai_dumps.py`	the fluid of a `visit_dump.*` (SAMRAI HDF5): MATLAB's `h5read`; Python's h5py	33
`read_vtk_legacy.m` / `read_vtk_legacy.py`	VisIt's legacy ASCII VTK exports (resampled grids, meshes, points) and IB2d's `.vtk` output	33
`read_exodus.m` / `read_exodus.py`	IBFE's Exodus files (Octave's `netcdf`; netCDF4)	33
`read_curve.m` / `read_curve.py`	VisIt's `.curve` files, several curves per file kept apart	29

Chapters 11 and 33 describe both versions of their readers (chapter 11: `read_hier_data`, `read_input`, `leaf_forces`; chapter 33: `samrai_dumps`, `read_vtk_legacy`, `read_exodus`), and chapter 29 runs both versions of `read_curve` on its curve files (the pick through time: two curves in one file). The `.m` files ran in Octave 8.4 except `samrai_dumps.m` (no `h5read` in Octave), which ran in MATLAB R2024a after two fixes (October 2026) and gives the same arrays as `samrai_dumps.py`.

TRANSLATING MATLAB TO PYTHON (1/3): INDICES, RANGES, SHAPES

```
x = 0:dx:1;              % includes 1
y = zeros(N,1);          % a column of N
for k = 1:N              % 1 ... N
    y(k) = sin(2*pi*x(k));
end
y(end)                   % the last one
fprintf(fid,'%d %1.16e\n', k-1, y(k));
```

```
x = np.arange(N + 1) * dx  # 0 ... 1: count, don't step
y = np.zeros(N)            # zeros(N) in MATLAB is N x N
for k in range(N):         # 0 ... N-1
    y[k] = np.sin(2*np.pi*x[k])
y[-1]                      # the last one
f.write('%d %1.16e\n' % (k, y[k]))
```

- `a:d:b` includes `b` when it lands on it (`0:0.1:0.3` has 4 points, the last 0.3); `np.arange(a, b, d)` is meant to stop before `b`, but with a float step round-off can add a point or lose one: `np.arange(1, 1.3, 0.1)` ends at 1.3. Build the grid from an integer count, `np.linspace(a, b, n)` or `a + d*np.arange(n)`.
- MATLAB indexes from 1, Python and IBAMR's structure files from 0 (IB2d's MATLAB files from 1, pyIB2d's from 0): shift the index when it is written into a file, not the loop.

The ports of this series (the support exercises, the generators, chapter 4c's IB2d ports).

TRANSLATING MATLAB TO PYTHON (2/3): ARRAYS, NUMBERS, FILES

- Arrays: `A*B` is a matrix product in MATLAB and element-wise in NumPy (`A @ B` for the product); `A'` is the conjugate transpose (`A.T` for real arrays). MATLAB's arrays are stored by columns, NumPy's by rows: when you reshape a flat list of values read from a file, say which order the file used (`order='F'` for column by column).
- Integer division: `floor(n/2)` in MATLAB, `n // 2` in Python; `/` on integers gives a float in Python 3.
- Files: `fprintf`'s conversions (`%d`, `%e`, `%f`, `%g`) mean the same in Python's `%`-formatting, with two differences: `fprintf` repeats its format over every element of an array (`sprintf('%d', [1 2 3])` is `123`; in Python, loop or use `np.savetxt`), and `%d` of a number with a fraction, 2.5, prints `2.500000e+00` in MATLAB, `2.5` in Octave and `2` in Python. Write the same format and the files can be compared byte for byte (slide 14).

The ports of this series; NumPy for MATLAB users, https://numpy.org/doc/stable/user/numpy-for-matlab-users.html.

TRANSLATING MATLAB TO PYTHON (3/3): PLOTS, FUNCTIONS AND SCRIPTS

- Plots: save to a file (`plt.savefig`) and open a window only when asked, since a cluster job has no display: the exercises' Python versions save a PNG and open a window with `--show`; `channel_flow.py` plots only with `--plot`. MATLAB and Octave can save a figure without showing it too (`figure('Visible','off')`, then `print` or `saveas`; `octave-cli` did so here without a display, through gnuplot, which Octave needs for any figure when there is no display). Of the MATLAB generators only IB/02's `Channel_Flow.m` still opens a figure; the ConstraintIB ones have their `plot` lines commented out, so that `octave-cli` runs them even with neither a display nor gnuplot (slide 6).
- Functions and scripts: a MATLAB function file becomes a Python function plus a `if __name__ == "__main__":` block that reads `sys.argv`, as `heat_equation.py N T` does for `heat_equation(N,T)`.

The ports of this series; the generators' and exercises' `README.md` files.

EXERCISES

1. Run `heat_equation` with `19 100`, `19 70` and `19 50` in MATLAB or Octave and in Python; the `.m` file prints nothing, so add `max(abs(up - exp(-4*pi^2*t)*sin(2*pi*x)))` after its loop. Compare the largest errors, and explain why the languages (and one language on two systems) differ most in the unstable case.
2. Find the largest Δt for which `stiff_ode_example` decays without changing sign, and the largest for which it stays bounded; check them against $1/15$ and $2/15$.
3. Run IB/03's two generators and compare their files with `cmp` (slide 14); then IB/01's, and compare each `.vertex` with the shipped `rubber_band_512.vertex` (MATLAB's): measure the largest difference as numbers. On one Linux machine Octave and Python agree byte for byte; across systems they do not.
4. Write the Python version of a small MATLAB script of your own; test it the same way.
5. Read the same frame of `hier_data_IB2d/X.*` (from a run of IB/03 or another IB example: the repository ships no output) with `read_hier_data.m` and `hier_data.py`, and check that the positions agree.

SUMMARY AND SOURCES

- 26 MATLAB scripts in the series' examples (`legacy/` aside), each with a Python version: 4 course exercises, 7 readers, 15 structure generators. Most generator pairs write identical files; the rest (IB/01, 04, 08, 11) differ by round-off in the last digit, which depends on the system, as their READMEs say. `samrai_dumps.m` and the two geometry checkers were fixed in October 2026.
- MATLAB R2024a, Octave 8.4 or Python 3.12: use what you have. Python's versions save their plots and run on a cluster.
- When translating: count grid points instead of stepping, shift indices only when writing files, write the same number formats, and compare the outputs.

Sources: `examples/support/matlab-exercises`, `examples/analysis` and every example's `README.md` (October 2026); the course originals in `originals/MATLAB codes/`; NumPy's guide for MATLAB users.

> The figures of this appendix were made by the Python versions, run for it.